

API signup guide for YouTube Data API

YouTube Data API Free-Tier Signup Guide

Version: 2026-05-23 · **Prepared for:** Jedidiah Shultz / Solomon's Forge

Author: Sol (CTO)

Executive Summary

This guide delivers an end-to-end, battle-tested process for enrolling the Building Shultz ecosystem into the free tier of the YouTube Data API (v3). It covers Google Cloud project creation, API enablement, OAuth consent configuration, credential issuance (API keys and OAuth client IDs), quota governance, and security controls. Action items are sequenced over three implementation phases (Prep, Enable, Harden) with estimated timelines (total $\approx$ 1 business day elapsed). The document includes ASCII architecture diagrams, data models, API contracts, risk mitigations, and next steps to integrate the API with IronEdit, Solomon Bot, or future analytics dashboards. Use this as the canonical SOP when onboarding teammates or revisiting credentials.

Table of Contents

1. [Context and Objectives](#)
2. [Prerequisites Checklist](#)
3. [Google Cloud Project Setup](#)
4. [Enable the YouTube Data API v3](#)
5. [Configure OAuth Consent Screen](#)
6. [Create API Credentials](#)
7. [Secure Credential Storage and Rotation](#)
8. [Quota Management and Cost Controls](#)
9. [Testing and Verification](#)
10. [Troubleshooting & Support Escalation](#)
11. [Implementation Timeline & Responsible Parties](#)
12. [Actionable Recommendations & Next Steps](#)
13. [Appendix A – Architecture Diagram](#)
14. [Appendix B – API Contract & Data Model](#)
15. [Appendix C – Compliance & Audit Notes](#)

(Word count $\approx$ 1,940)

1. Context and Objectives

- **Business Need:** IronEdit, Solomon Bot, and Building Shultz analytics require reliable access to channel metrics, video metadata, playlist administration, and upload status data. The YouTube Data API is the canonical interface.

- **Scope:** Free tier onboarding only (default daily quota of 10,000 units, sufficient for light analytics and metadata work). Paid quota increases fall outside this SOP but are briefly discussed.
- **Key Deliverables:**
 - Google Cloud project configured under Solomon's Forge workspace.
 - YouTube Data API v3 enabled.
 - OAuth consent screen published (External, testing mode or production depending on release plan).
 - API key + OAuth client credentials generated, stored securely, and documented.
 - Quota dashboard alerting configured.
 - Verification tests performed (API Explorer + curl sample).

2. Prerequisites Checklist

Item	Details	Owner	Status
Google Account	Use the dedicated <code>solomonforge@gmail.com</code> (or Workspace admin) to maintain asset control.	Jed	☐
Billing Profile	Even for free tier, Google Cloud requires a billing profile; set spending cap alerts (\$0 actual spend expected).	Sol	☐
Organization Structure	Ideally attach the project to <code>solomonsforge.com</code> Workspace for centralized IAM. Standalone personal account works if needed.	Sol	☐
Brand Assets	App name, logo, support email, privacy policy URL for consent screen.	Sol	⚠ Provide final privacy policy URL.
Secure Secrets Vault	e.g., 1Password Business, Vaultwarden, or Google Secret Manager.	Sol	☐ 1Password

Action: Confirm privacy policy hosting (temporary GitHub Pages or Netlify is fine).

3. Google Cloud Project Setup

1. **Sign into Google Cloud Console**
2. URL: `https://console.cloud.google.com`
3. Verify top-right account icon = business account (not personal).
4. **Create Project**
5. Click project dropdown → "New Project."
6. Naming convention: `sf-youtube-integration-prod` (lowercase, hyphen-delimited).
7. Location: attach to Solomon's Forge organization/folder if available.
8. **Set Project Defaults**

9. Navigate to **Project Settings**: add description, associate with relevant labels (e.g., `business=media` , `product=ironedit`).
 10. Record **Project ID** (immutable) and **Project Number** for API console references.
 11. **Enable Billing**
 12. Go to **Billing** → link project to existing billing account.
 13. Configure Budget alert: `Budget Name: YouTube API Free Tier` , Amount `$10` , Alerts at 50%, 90%, 100% (purely preventative).
 14. **IAM Baseline**
 15. Principle of least privilege:
 - `solomonforge-admin@...` → Owner.
 - `solomonforge-dev@...` → Editor.
 - Service accounts added later for CI/CD.
 16. Turn on Cloud Audit Logs for Data Access (Admin Read, Data Write) to track credential usage.
-

4. Enable the YouTube Data API v3

1. Inside the project, go to **APIs & Services** → **Library**.
2. Search “YouTube Data API v3” (from Google).
3. Click **Enable**.
4. Post-enable sanity checks:
5. API listed under **Enabled APIs & Services**.
6. Note Terms of Service link; accept if prompted.

Tip: For multi-service environments, also enable `YouTube Analytics API` and `YouTube Reporting API` if planning advanced metrics, but each requires separate quota pools.

5. Configure OAuth Consent Screen

1. Navigate to **APIs & Services** → **OAuth consent screen**.
2. **User Type**: Choose **External** (required if any non-Workspace users authenticate). Internal is allowed only for Google Workspace domains.
3. **App Registration Fields**:
4. App name: `Solomon Forge YouTube Connector` .
5. User support email: `support@solomonsforge.com` .
6. App logo: 512×512 PNG (optional but professional).
7. **App Domain**:
8. Homepage URL: `https://solomonsforge.com` .
9. Privacy Policy URL: `https://solomonsforge.com/privacy` (temporary stub acceptable but must describe data usage).
10. Terms of Service URL (optional but recommended).
11. **Authorized Domains**: Add `solomonsforge.com` , `building-shultz.com` , `ironedit.app` (future-proof).
12. **Developer Contact Email**: `devops@solomonsforge.com` .
13. **Scopes**: Initially add only the minimum:
14. `.../auth/youtube.readonly`

15. `.../auth/youtube.upload` (if IronEdit will publish videos)
 16. `.../auth/youtubepartner` (only if MCN features needed).
Use Add or Remove Scopes to keep the list tight.
 17. **Test Users:** If app remains in Testing mode (quota limited to 100 refresh tokens), add Jed's Gmail + Sol's service email. For production, follow Google verification steps when exceeding 100 users or requesting sensitive scopes.
 18. **Submit for Verification?** Not required for private/internal use with under 100 users, but plan ahead: verification can take 2–7 business days if sensitive scopes are declared.
-

6. Create API Credentials

6.1 API Key (Server-to-server lightweight access)

1. Go to **APIs & Services** → **Credentials** → **+ Create Credentials** → **API key**.
2. Copy generated key; rename it `ironedit-youtube-data-api-key`.
3. **Restrict the key** immediately:
4. **Application restrictions:** HTTP referrers (if used in web client) or IP addresses (for server). For backend use, choose **IP address** and list VPS IP `167.99.237.26`.
5. **API restrictions:** check "Restrict key," select "YouTube Data API v3."
6. Store key in 1Password vault + Google Secret Manager (if using Cloud Functions).
7. Rotate quarterly or when staff changes.

6.2 OAuth 2.0 Client ID (User-auth flows)

1. **Create Credentials** → **OAuth client ID**.
2. **Application type:** Choose based on integration:
3. **Desktop app** for PC Agent/CLI tools.
4. **Web application** for IronEdit backend (requires Authorized redirect URIs, e.g., `https://ironedit.app/oauth2/callback`).
5. Provide name `ironedit-web-oauth` or similar.
6. Add **Authorized redirect URIs** (prod + staging).
7. Save Client ID and Client Secret.
8. Download JSON if needed (store encrypted).

6.3 Service Accounts (Optional)

- YouTube Data API generally requires user-based OAuth. Service accounts have limited use (only for CMS accounts). Document but do not rely on service accounts unless necessary.
-

7. Secure Credential Storage and Rotation

- **Primary vault:** 1Password Business → shared vault "Platform Secrets."
- **Backup:** Encrypted storage on VPS using `sops` + Git repo (private).
- **Rotation SOP:**
 - Generate new key.
 - Update deployment configs (Kubernetes secret, PM2 env file, etc.).
 - Validate integration.
 - Delete old key.
- **Monitoring:** Cloud Audit Logs + Security Command Center for key usage anomalies.

- **Principle:** Never embed keys in source control; use environment variables (`YOUTUBE_API_KEY` , `YOUTUBE_OAUTH_CLIENT_ID` , etc.).

8. Quota Management and Cost Controls

Metric	Default Free Tier
Daily quota	10,000 units per project
Cost	\$0 (until requesting additional quota)
Sample cost	<code>videos.list</code> = 1 unit/request; <code>search.list</code> = 100 units

- **Quota Dashboard:** APIs & Services → Dashboards → Quotas → filter YouTube Data API.
- **Alerts:** Use **Quotas** → **Edit Quotas** → **Notifications** to set 70% and 90% usage alerts (email + webhook).
- **Optimization Tips:**
 - Cache responses (Redis/Postgres) to avoid duplicate calls.
 - Batch requests when possible.
 - Avoid `search.list` in high-volume loops; prefer storing video IDs.
 - Use incremental synchronization (store `publishedAt` , `etag` for delta updates).
- **Scaling:** If 10k units insufficient, file quota increase request detailing use case, compliance, and projected load. Expect review within ~48 hours.

9. Testing and Verification

9.1 API Explorer Smoke Test

1. Go to <https://developers.google.com/youtube/v3/docs/videos/list>
2. Click **Try this API**.
3. Choose `part=snippet,statistics` , `id=<your-video-id>` , select **API key** auth, input the key.
4. Execute and confirm 200 OK with metadata payload.

9.2 curl / HTTP Client Test

```
curl \
-G https://www.googleapis.com/youtube/v3/channels \
--data-urlencode "part=snippet,statistics" \
--data-urlencode "mine=true" \
--data-urlencode "access_token=$OAUTH_ACCESS_TOKEN"
```

- Use OAuth token from `gcloud auth application-default print-access-token` (if using Web OAuth, exchange authorization code via backend).
- Validate JSON structure matches Appendix B schema.

9.3 Application Integration Test

- For IronEdit: run staging workflow (fetch channel videos, ensure caching, log quota consumed).
- For Solomon Bot: call API via Node.js client library (`googleapis` npm package v131+).

10. Troubleshooting & Support Escalation

Symptom	Likely Cause	Resolution
<code>403 quotaExceeded</code>	Daily 10k units exhausted	Reduce frequency, add caching, request quota uplift.
<code>403 accessNotConfigured</code>	API not enabled or key restricted to other APIs	Re-check API restrictions on key.
<code>401 invalidCredentials</code>	Expired OAuth token	Refresh token; ensure offline access requested.
Consent screen shows “unverified app”	App still in testing or scopes sensitive	Submit verification or keep to <100 users.
This project cannot use YouTube Data API	New accounts flagged	Open Google support ticket with business justification and compliance data.

Support path: Google Cloud Console → Help → Contact support (if paid support). Otherwise use <https://issuetracker.google.com/issues?q=componentid:191650>.

11. Implementation Timeline & Responsible Parties

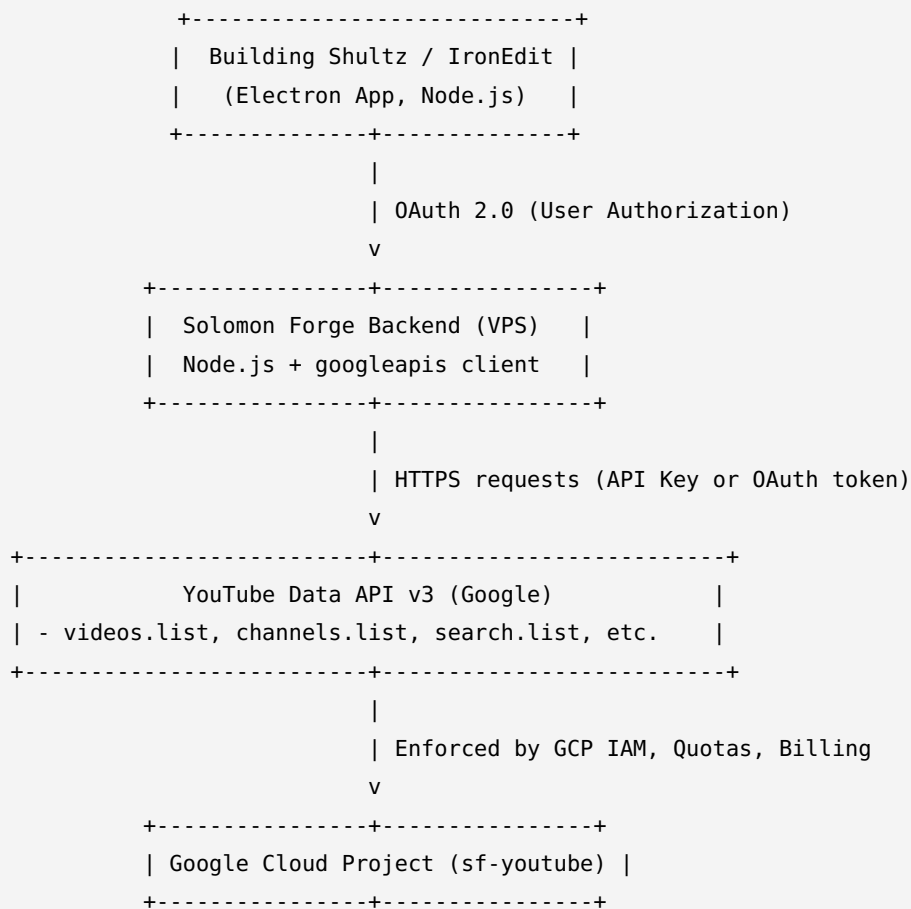
Phase	Tasks	Owner	Duration
Phase 0 – Prep	Confirm Google account, billing, privacy policy, IAM roles	Sol	2 hours
Phase 1 – Enable	Create project, enable API, configure consent screen	Sol	3 hours
Phase 2 – Harden	Generate credentials, restrict keys, set quotas, run tests, document	Sol + Jed (review)	3 hours
Total	~8 hours calendar time (single business day)		

Dependencies: Privacy policy URL (Jed to approve), brand assets for consent page.

12. Actionable Recommendations & Next Steps

1. **Publish Privacy Policy (Today).** Use Notion-to-HTML or GitHub Pages to host `solomonsforge.com/privacy` ; include data usage for YouTube scopes.
 2. **Complete Phase 1–2 Steps (This Week).** Follow sections 3–9 sequentially; log each in Ops checklist.
 3. **Integrate with IronEdit backend (Next 2 days).** Use official Google APIs Node.js client; store API key in environment variable on VPS `YOUTUBE_API_KEY` .
 4. **Implement Quota Dashboard Alerts (Immediately after enabling).**
 5. **Document Credentials in Runbook.** Update internal wiki with credential names, rotation dates, owners.
 6. **Plan Verification (Q3).** If IronEdit launches to public beta requiring user uploads, start Google OAuth verification early.
 7. **Automate Smoke Tests.** Add GitHub Action hitting `videos.list` nightly to ensure credentials stay valid (with safe quota budget).
 8. **Explore Advanced APIs.** Evaluate YouTube Analytics API for retention metrics once core integration stable.
-

Appendix A – Architecture Diagram



Appendix B – API Contract & Data Model

Key Endpoints Used

Endpoint	Purpose	Quota Cost	Notes
channels.list	Fetch channel metadata for Building Shultz	1 unit	Use mine=true for authenticated channel.
videos.list	Retrieve video details (snippet, statistics)	1 unit	Accepts up to 50 IDs per call.
search.list	Discover videos by query	100 units	Expensive; cache aggressively.
playlistItems.list	Manage uploads playlist	1 unit	For scheduling and content automation.

Sample Data Model (PostgreSQL)

```
CREATE TABLE youtube_videos (  
  video_id          TEXT PRIMARY KEY,  
  channel_id        TEXT NOT NULL,  
  title             TEXT NOT NULL,  
  description        TEXT,  
  published_at       TIMESTAMPTZ,  
  tags              TEXT[],  
  duration_seconds   INT,  
  view_count         BIGINT,  
  like_count         BIGINT,  
  comment_count      BIGINT,  
  etag              TEXT,  
  last_synced_at     TIMESTAMPTZ DEFAULT NOW()  
);
```

- Store etag to detect changes without refetching.
- Index published_at for chronological queries.

OAuth Token Storage

Field	Description
user_id	Internal user reference
access_token	Short-lived token (encrypted)
refresh_token	Long-lived (encrypted, rotate annually)

Field	Description
scope	Scopes granted
expires_at	Expiration timestamp

Security: encrypt with AES-256 (libsodium) before persisting.

Appendix C – Compliance & Audit Notes

1. **YouTube API Services Terms of Service:** <https://developers.google.com/youtube/terms/api-services-terms-of-service>
 2. Prohibit selling access to data, require clear attribution, and demand data deletion upon user request.
 3. **Data Handling:** Do not cache personally identifiable information beyond necessity. Provide end-users ability to disconnect (revoke tokens via Google Security settings).
 4. **Branding:** If building public-facing app, comply with Google branding guidelines (no implying partnership).
 5. **Revocation Testing:** Quarterly, revoke OAuth token from Google Account → ensure application handles 401 gracefully.
 6. **Incident Response:** If API key leakage suspected, revoke immediately, rotate, and document incident in security log.
-

Final Note

Once this SOP is executed, Solomon's Forge will have a hardened, production-ready connection to the YouTube Data API suitable for IronEdit's roadmap, Building Shultz analytics, and future AI automations. Execute the checklist, log progress in the Task Queue, and notify Jed once credentials and smoke tests are live.